

Final Project Report

Performance Analysis of Parallel GMRES Solvers for 2D Fluid Flow on High-Performance Clusters

Sarju Poudel

Iowa State University
Mechanical Engineering Department
Numerical Analysis of High Performance Computing
Math 5250: Spring 2026

May 9, 2026

Abstract

This project investigates the performance of parallel computing methods for two-dimensional incompressible fluid flow simulations on high-performance computing (HPC) systems. The governing equations were solved using a finite-difference based approach with iterative numerical methods.

Three implementations were developed and compared: serial execution, shared-memory parallelization using OpenMP, and distributed-memory parallelization using MPI. The computational domain was discretized into a structured grid, and parallel efficiency was analyzed using different processor configurations and problem sizes.

The results show that OpenMP provides strong performance improvements on shared-memory systems, while MPI enables distributed computation but introduces communication overhead that limits scalability in some cases. Overall, the study highlights the trade-off between computation and communication in parallel CFD simulations and the importance of efficient domain decomposition for scalable performance.

Keywords: CFD, GMRES, MPI, OpenMP, Navier–Stokes, High-Performance Computing, Parallel Computing, Fluid Flow Simulation

1 Introduction

Computational Fluid Dynamics (CFD) is a fundamental tool for analyzing fluid flow in engineering and science, with applications in aerodynamics, weather prediction, and biomedical systems. These problems involve solving complex partial differential equations over large computational domains, making them computationally expensive and memory intensive. As problem sizes increase, high-performance computing (HPC) becomes essential for practical and efficient simulations.

In incompressible flow, the Navier–Stokes equations are coupled with a pressure Poisson equation, which often represents the most computationally expensive part of the solution process. Efficient numerical solvers are therefore critical for achieving good performance.

As computational demands grow, traditional serial methods become inefficient, motivating the use of parallel computing. Shared-memory parallelization using OpenMP enables concurrent execution of loops using multiple threads within a single node, while MPI enables distributed-memory computing by dividing the domain across multiple nodes using domain decomposition. In MPI, ghost-cell exchanges are required to maintain consistency across subdomain boundaries. While MPI provides scalability, its performance is strongly influenced by communication overhead, load balancing, and problem size.

This project implements and compares serial, OpenMP, and MPI versions of a GMRES-based solver for two-dimensional incompressible flow. The focus is on evaluating performance, scalability, and communication costs across different grid sizes and computing configurations on both local systems and HPC clusters. Performance is assessed using wall-clock time, convergence behavior, and scalability to understand the trade-offs between computation and communication in parallel CFD simulations.

1.1 Mathematical Formulation

The incompressible Navier–Stokes equations for 2D flow are:

$$\rho \left(\frac{\partial u}{\partial t} + (u \cdot \nabla)u \right) = -\nabla p + \mu \nabla^2 u + f, \quad \nabla \cdot u = 0 \quad (1)$$

where $u = (u_x, u_y)$ is velocity, p is pressure, and ρ, μ are density and viscosity. The system is typically solved using a projection method that decouples velocity and pressure.

1.2 Numerical Discretization

The advection and diffusion terms are discretized as:

$$(u \cdot \nabla)v \approx u^n \frac{\partial v^n}{\partial x} + v^n \frac{\partial v^n}{\partial y}, \quad (2)$$

$$\nu \nabla^2 u^n \approx \nu \left(\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right) \quad (3)$$

An intermediate velocity is computed as:

$$u^* = u^n - \Delta t(\text{advection}) + \Delta t(\text{diffusion}) + \Delta t(\text{forcing}) \quad (4)$$

This yields a linear system after spatial discretization.

1.3 Pressure Poisson Equation

The pressure equation becomes:

$$\nabla^2 p = \frac{\rho}{\Delta t} \nabla \cdot u^* \quad (5)$$

with discrete forms:

$$\nabla \cdot u^* \approx \frac{u_{i+1,j}^* - u_{i-1,j}^*}{2\Delta x} + \frac{v_{i,j+1}^* - v_{i,j-1}^*}{2\Delta y}, \quad (6)$$

$$\nabla^2 p_{i,j} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} \quad (7)$$

After discretization, the system becomes:

$$Ap = b \quad (8)$$

1.4 GMRES Solver

GMRES solves $Ap = b$ by building a Krylov subspace:

$$\mathcal{K}_k(A, r_0) = \text{span}\{r_0, Ar_0, A^2r_0, \dots, A^{k-1}r_0\} \quad (9)$$

where $r_0 = b - Ap_0$. At each iteration it minimizes:

$$\|b - Ap_k\|_2 \quad (10)$$

until:

$$\frac{\|r_k\|}{\|r_0\|} < \text{tol} \quad (11)$$

1.5 Domain Discretization

The physical domain was discretized into a structured computational grid using finite differences, as shown in Figure 1. Each computational cell stores the velocity components (u, v) and pressure p .

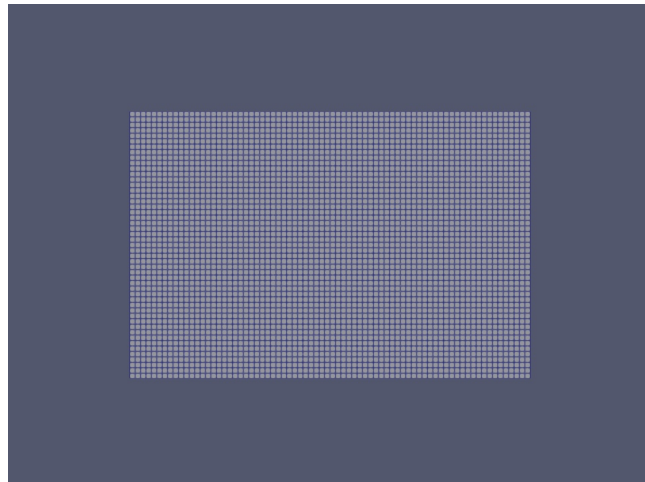


Figure 1: Computational domain discretization used for the CFD simulation.

Structured finite-difference discretization provides a simple and efficient framework for implementing stencil-based numerical operations. The regular arrangement of computational cells also facilitates parallel decomposition and communication strategies for distributed-memory computing.

2 Methods

This section presents the computational methodology used in this study, including the parallel implementation strategies, domain decomposition techniques, numerical algorithms, and simulation parameters employed for incompressible CFD simulations using GMRES-based solvers. Three computational approaches were developed and compared in this work: serial execution, shared-memory parallelization using OpenMP, and distributed-memory parallelization using MPI. These implementations were designed to evaluate computational performance, scalability, and communication overhead across different high-performance computing (HPC) architectures.

2.1 Serial Implementation

The serial implementation solves the entire CFD domain using a single processor core without parallelization. This implementation serves as the baseline reference for evaluating the speedup and scalability of the parallel implementations.

2.2 OpenMP Parallelization

OpenMP was used for shared-memory parallelization across multiple threads, with parallel execution applied to key computational loops such as matrix-vector multiplication, velocity updates, and residual calculations. It benefits from low communication overhead since all threads share the same memory space, but performance is limited by memory bandwidth, synchronization costs, and the number of available CPU cores.

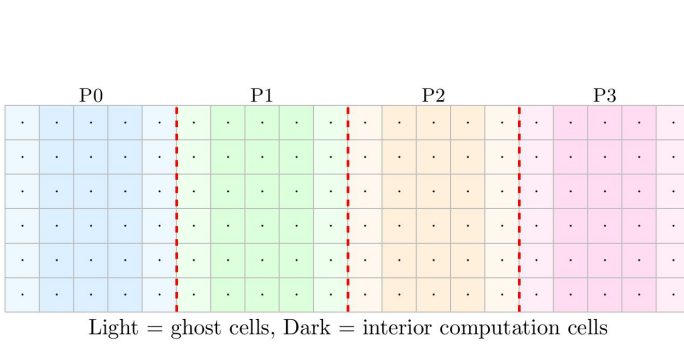
2.3 MPI Parallelization

MPI was used for distributed-memory parallelization via domain decomposition, where the computational domain was divided among multiple ranks, each handling its own subdomain. Each process stores local grid data, performs computations on the governing equations, and exchanges ghost-cell information with neighboring processes. This communication ensures consistency across subdomains during stencil-based finite-difference operations.

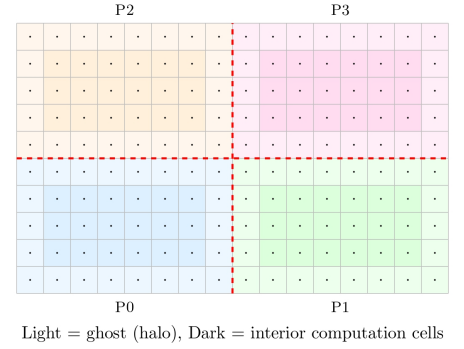
2.3.1 Domain Decomposition and Ghost Cells

Efficient domain decomposition is essential for distributed-memory CFD simulations. In two-dimensional fluid flow problems, either one-dimensional or two-dimensional MPI decomposition can be used; however, this work implements only one-dimensional decomposition. In this approach, the domain is split along a single spatial direction, so each MPI rank communicates with only two neighbors, which keeps communication relatively simple but can limit load balancing and scalability as the number of processes increases. In contrast, two-dimensional decomposition partitions the domain in both spatial directions, producing smaller subdomains, improved load distribution, and better scalability for large processor counts and problem sizes.

Ghost cells, also referred to as halo regions, were introduced around each local subdomain to facilitate stencil computations near subdomain boundaries. Ghost Cells store neighboring boundary values and are updated using MPI communication.



(a) 1D MPI Domain Decomposition
with Ghost Cells Visualization



(b) 2D MPI Domain Decomposition
with Ghost Cells Visualization

Figure 2: Comparison of 1D and 2D MPI domain decomposition with ghost cell regions

2.4 Time-Stepping Algorithm

The incompressible CFD solver uses a projection-based time-stepping scheme in which velocity and pressure are updated sequentially at each time step to enforce incompressibility. The algorithm proceeds as follows:

1. Apply predictor boundary conditions and exchange ghost-cell data
2. Compute intermediate velocity field
3. Assemble pressure Poisson right-hand side
4. Solve pressure equation using GMRES
5. Correct velocity field and apply final boundary conditions
6. Output diagnostics

2.5 Boundary Conditions

The simulations use standard boundary conditions for incompressible flow, including a prescribed inlet velocity, no-slip walls ($u = v = 0$), and a pressure extrapolation condition at the outlet to allow smooth flow exit. MPI ghost-cell communication is handled separately to maintain consistency across subdomain interfaces in the parallel decomposition.

2.6 Simulation Parameters

Table 1: Comparison of simulation parameters for local machine and HPC cluster runs.

Parameter	Local Machine	HPC Cluster
Grid Size	75×50	1536×1024
Domain Size	$L_x = 1.5, L_y = 1.0$	$L_x = 1.5, L_y = 1.0$
Viscosity	$\nu = 0.05$	$\nu = 0.05$
Density	$\rho = 1.0$	$\rho = 1.0$
Time Step	$\Delta t = 10^{-5}$	$\Delta t = 10^{-5}$
Total Steps	7500	7500
Tolerance	10^{-4}	10^{-4}
Max GMRES Iterations	2000	2000

3 Results

3.1 Pressure Field

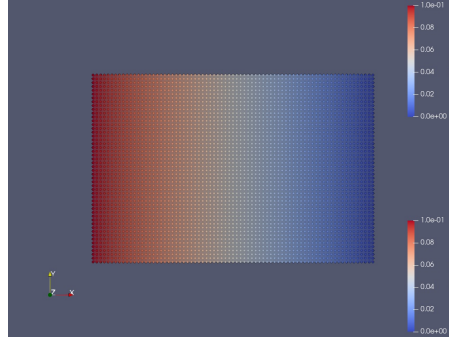


Figure 3: Pressure distribution across the computational domain.

The pressure field demonstrates smooth spatial variation throughout the computational domain, indicating stable numerical convergence of the pressure Poisson equation.

3.2 Velocity Field

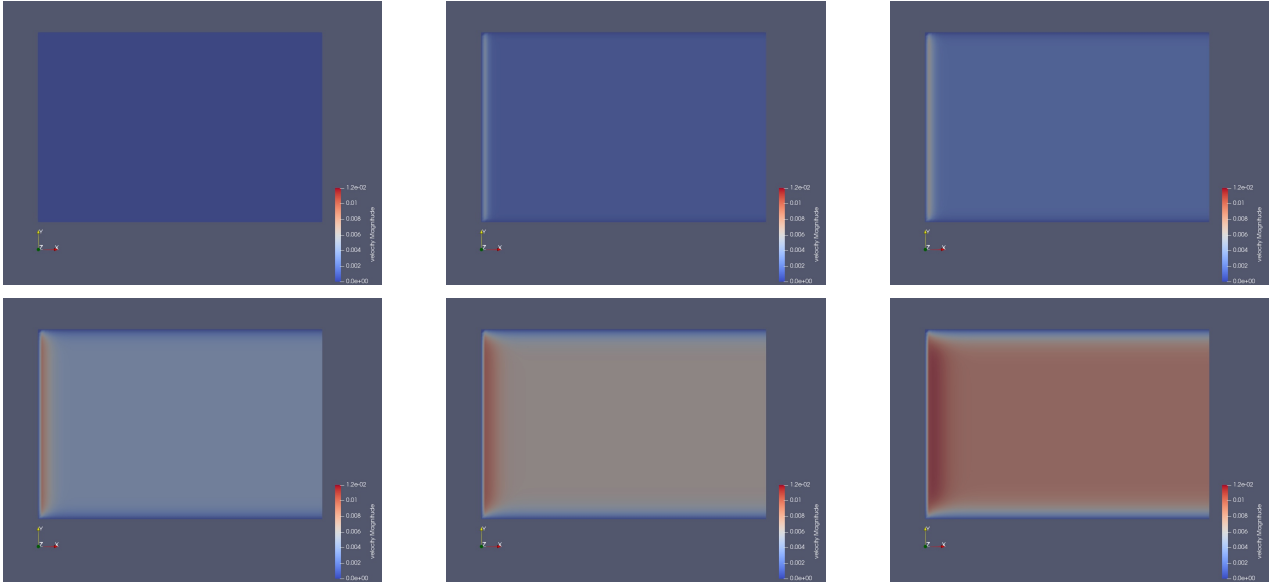


Figure 4: Time evolution of velocity field at selected time steps.

3.3 GMRES Residual Convergence

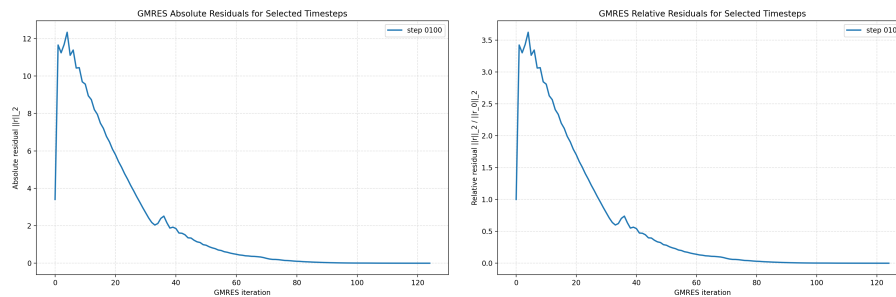


Figure 5: Absolute and relative GMRES residual convergence histories.

The GMRES solver demonstrated stable convergence behavior throughout the simulation. Residual reduction confirms the effectiveness of the iterative solver for the pressure Poisson system.

3.4 Performance Analysis

3.4.1 Local Machine Performance

Table 2: Execution time on local machine including Serial, OpenMP, and MPI configurations.

Case	Total Time (s)	Operator Time (s)	GMRES Time (s)
Serial	17.379	7.148	16.780
OpenMP (1)	22.412	10.158	21.687
OpenMP (2)	18.398	8.878	17.904
OpenMP (4)	15.036	7.419	14.667
OpenMP (6)	14.801	7.393	14.457
MPI (1)	17.388	7.046	16.907
MPI (2)	12.112	4.338	11.811
MPI (4)	9.209	2.863	9.012
MPI (6)	12.400	2.804	12.127

For small domains, parallel overhead limits performance gains. MPI showed optimal performance near four processes, while excessive thread or process counts introduced additional communication and synchronization overhead.

3.4.2 HPC Cluster Performance

Table 3: Execution time on Nova for small domain simulations.

Case	Total (s)	Operator (s)	GMRES (s)
Serial	19.127	8.159	18.502
OpenMP (1)	20.912	9.313	20.222
OpenMP (2)	15.924	7.659	15.457
OpenMP (4)	13.937	7.003	13.575
OpenMP (6)	14.218	7.309	13.881
OpenMP (8)	15.543	8.362	15.202
OpenMP (16)	22.401	12.427	21.980
OpenMP (32)	37.497	20.469	36.876
OpenMP (64)	49.384	24.892	48.588
MPI (1)	18.596	8.186	18.039
MPI (2)	12.724	5.015	12.384
MPI (4)	9.504	3.336	9.286
MPI (6)	8.899	2.711	8.717
MPI (8)	8.122	2.402	7.963
MPI (16)	7.819	1.928	7.685
MPI (32)	10.925	2.016	10.752

Table 4: Execution time on Nova cluster for large domain simulations.

Case	Total Time (s)	Operator Time (s)	GMRES Time (s)
Serial	1374.062	907.681	1372.914
OpenMP (1)	1378.914	913.710	1377.394
OpenMP (2)	740.541	489.124	739.696
OpenMP (4)	443.820	276.921	443.086
OpenMP (8)	222.771	138.435	222.332
OpenMP (16)	113.318	70.121	113.042
OpenMP (32)	59.168	36.555	58.893
OpenMP (64)	35.165	20.873	34.536
MPI (8)	2696.677	1199.057	2660.404
MPI (16)	1747.367	517.656	1728.287
MPI (32)	984.383	284.712	973.672
MPI (64)	877.651	277.549	867.017

OpenMP demonstrated strong scalability on shared-memory systems. MPI simulations exhibited significant communication overhead, especially for pressure solver communication and ghost-cell exchange.

4 Discussion and Conclusion

Several important computational observations were identified during the study. Small computational domains do not benefit significantly from parallelization due to the relatively high cost of synchronization and commu-

nication compared to computation. In contrast, OpenMP shows strong performance scaling for larger domains because communication overhead remains minimal in shared-memory systems. MPI enables distributed-memory scalability, but its performance is limited by communication overhead from halo exchanges and global reductions in the solver, especially in multi-node configurations where inter-node latency further reduces efficiency. Overall, efficient ghost-cell communication and well-balanced domain decomposition are essential for achieving scalable performance in distributed CFD simulations.

This project developed and analyzed a CFD solver for incompressible two-dimensional flow using a GMRES-based approach, with three implementations: serial, OpenMP, and MPI, evaluated on both local and HPC systems. Results show that serial execution is inefficient for large-scale problems, while OpenMP provides strong scalability on shared-memory architectures. MPI enables scalability across distributed systems but requires careful optimization due to communication costs. The study highlights the importance of domain decomposition and ghost-cell communication for correctness and efficiency in parallel CFD simulations. Overall, OpenMP is highly effective for shared-memory systems, while MPI offers a flexible framework for large-scale simulations when communication and solver strategies are further optimized.

5 Limitations and Future Work

Several limitations were identified in the current implementation. MPI parallelization introduces communication overhead, and the gather-to-root output strategy limits scalability for large simulations. The lack of parallel I/O (e.g., MPI-IO or HDF5) further restricts performance at high resolutions, while GMRES communication patterns add additional inefficiencies. A hybrid MPI+OpenMP approach was also not explored.

Future improvements should focus on reducing communication overhead using non-blocking methods and better synchronization strategies. Hybrid parallelization with MPI and OpenMP could improve scalability across modern HPC systems, while GPU acceleration may further enhance performance. Implementing parallel I/O and using preconditioned GMRES solvers would also improve efficiency and convergence.

References

- [1] J. H. Ferziger and M. Perić, *Computational Methods for Fluid Dynamics*, Springer, 3rd Edition, 2002.
- [2] Y. Saad and M. H. Schultz, “GMRES: A Generalized Minimal Residual Algorithm for Solving Nonsymmetric Linear Systems,” *SIAM Journal on Scientific and Statistical Computing*, vol. 7, no. 3, pp. 856–869, 1986.
- [3] H. K. Versteeg and W. Malalasekera, *An Introduction to Computational Fluid Dynamics*, Pearson Education, 2nd Edition, 2007.
- [4] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message Passing Interface*, MIT Press, 1999.
- [5] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press, 2007.